Reviewer Pack — Minimal Rank of Free Probability Spaces vs. Distinguishing T…

Entry 8d4b03f81c95 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 02:49:34 UTC

Minimal Rank of Free Probability Spaces vs. Distinguishing Tensor Width for BP_ReadTwice

Entry ID: `8d4b03f81c95` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-24 02:49:34 UTC

1. Conjecture

Field A (mathematical branch): Free Probability Theory **Field B** (complexity object): Branching Program: read-once vs read-twice (BP)

Statement:

> ['For any given n-variable boolean function f, let P_f be a random read-twice BP that computes f with high probability. Then the minimal rank of its free probability space, denoted as RankFreeProb(P_f), is asymptotically bounded by RankFreeProb(P_f) = O(log(W_f)), where W_f is the width of the smallest BP computing f.', 'Equivalently, for all instances of boolean functions with n ≤ 40 variables, there exists a BP f^* with width W_f and free probability space rank RankFreeProb(f^*) such that RankFreeProb(P_f) / log(W_f) = Θ(1).', 'Furthermore, for the trivial read-twice BP computing the constant function 0, we have RankFreeProb(P_0) = Ω(n).']

Rationale (proposer's reasoning):

> ['Free probability theory provides a probabilistic framework that may encode structural complexity in BPs. If the rank of the free probability space associated with a BP is related to its width, it would suggest that certain aspects of BP complexity can be understood through probabilistic structures.', 'This connection could potentially reveal new insights into the inherent complexity of computing boolean functions using read-twice BPs.']

Taxonomy category: `BP_READTWICE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `7269ed5160fbee1d`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if RankFreeProb(P_f) / log(W_f) falls within [0.5, 2] for at least 80% of the seeds and 95% confidence interval, and falsified if any seed produces a ratio outside this range or any seed results in RankFreeProb(P_0) < n.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "free probability theory" AND "branching program" AND "read-once vs read-twice" AND "tensor width" - "minimal rank of free probability spaces" AND BP_readTwice AND "asymptotically bounded" - "RankFreeProb" AND "BP with width W_f" AND "0(1)" AND boolean functions

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.4s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def compute_width(f):
        n = len(f)
        width = 0
        for i in range(n):
            if f[i] != f[0]:
                width += 1
        return width

    def compute_free_probability_space_rank(f):
        n = len(f)
        rank = 0
        for i in range(2**n):
            if f[i % n] == f[(i + 1) % n]:
                rank += 1
        return rank

    n = random.randint(5, 40)
    f = generate_random_boolean_function(n)
    W_f = compute_width(f)
    RankFreeProb_P_f = compute_free_probability_space_rank(f)
```

```
    if W_f == 0:
        return {
            "metric_name": "Rank vs Width",
            "metric_value": None,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "Width is zero"
        }

    ratio = RankFreeProb_P_f / math.log(W_f)

    return {
        "metric_name": "Rank vs Width",
        "metric_value": ratio,
        "instances_tested": 1,
        "conjecture_holds": 0.5 <= ratio <= 2,
        "counterexample": ""
    }

if __name__ == "__main__":
    if len(sys.argv) > 1:
        seeds = [int(s) for s in sys.argv[1:]]
    else:
        primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73,
        seeds = random.sample(primes, 30)

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        mean_ratio = sum(r["metric_value"] for r in results) / len(results)
        support_fraction = 1.0
        RESULT = f"SUPPORTED mean={mean_ratio} std=0 support_fraction={support_fraction}"
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        counterexample = "Ratio outside [0.5, 2]"
        RESULT = f"FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing

    print(RESULT)
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test timed out before producing data, which means it was unable to verify the conjecture's conditions. | next: Re-run the test with increased time limits or consider alternative methods for verifying the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11300
2	propose	ollama_remote	glm4:latest	0	0	13874
3	propose	ollama_remote	glm4:latest	0	0	13798
4	preregistration	ollama_remote	glm4:latest	0	0	6465
5	novelty	ollama_remote	glm4:latest	0	0	5212
6	novelty	ollama_remote	glm4:latest	0	0	5343
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14482
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9617
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8784
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8381
11	judge	ollama_remote	glm4:latest	0	0	13329

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 110584 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/8d4b03f81c95.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/8d4b03f81c95.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/8d4b03f81c95.tar.gz` (if generated)